

Detection of Rhythmic Patterns in Acoustic Peaks (V2)

Zachary Weinfeld

June 10, 2025

Purpose

This algorithm aims to detect recurring rhythmic patterns in a sequence of detected acoustic peaks. It evaluates every possible candidate interval derived from peak differences, tests each rhythm statistically, and consolidates similar patterns while removing weak or redundant ones. The result is a clean list of dominant intervals that likely represent true underlying rhythmic structure in the data.

Comparison with V1 Algorithm

Unlike the original version (V1), which clustered time differences before evaluating rhythmic structure, the new algorithm examines each rhythm candidate directly. It retains only those that achieve statistically significant alignment with the observed peaks and groups similar candidates to remove redundancy. These enhancements improve both precision and interpretability in detecting dominant rhythmic patterns.

Plain English Overview of Algorithm

0. Estimate Noise Level

Before any rhythm analysis, the algorithm estimates the natural variability in peak timing and amplitude. This standard deviation is used to compute a margin of tolerance when evaluating rhythmic alignment and to calibrate statistical tests later in the process.

1. Generate Rhythm Candidates

Every pair of peaks is used to define a potential rhythm interval $d = x_j - x_i$, anchored at x_i . Only differences that allow a sufficient number of repetitions (typically based on $\sqrt{n}$) within the signal duration are retained.

2. Count Hits Per Candidate

For each candidate, the algorithm walks forward in time from the anchor using $t_k = \text{anchor} + k \cdot d$, checking whether predicted steps align closely with real peaks, both in the x and y direction. Each match is a “hit,” and both the number of hits and total prediction attempts (tries) are recorded.

3. Prune Candidates via Binomial Test

The algorithm tests each candidate using a one-sided binomial hypothesis test. It asks: “Is the observed hit rate significantly better than random chance?” Candidates that fail this test (after adjusting for multiple comparisons) are removed.

4. Group Similar Candidates

Candidates that share a large number of overlapping hits are grouped using a similarity metric (Szymkiewicz–Simpson coefficient). Within each group, the strongest candidate is retained and others are marked as absorbed.

5. Final Filtering by Support

As a final step, candidates that don’t have enough total support (from both their own hits and absorbed ones) are removed. This ensures the algorithm outputs only the most robust rhythmic patterns.

Algorithm 1 Detection of Rhythmic Patterns in Acoustic Peaks

```
1: Input: Sorted peak times  $x = [x_1, x_2, \dots, x_n]$ , amplitudes  $y = [y_1, y_2, \dots, y_n]$ , total duration  $T$ 
2: Output: Filtered list of dominant rhythmic candidates

3: Step 0: Estimate Noise Level
4: Compute timing variability:  $\sigma_x \leftarrow \text{estimate\_std}(x)$ 
5: Compute amplitude variability:  $\sigma_y \leftarrow \text{estimate\_std}(y)$ 
6: Compute margins:  $m_x \leftarrow z_\alpha \cdot \sigma_x$ ,  $m_y \leftarrow z_\alpha \cdot \sigma_y$ 
7: Compute null probability:  $p_{\text{null}} \leftarrow \min(1, \frac{2 \cdot m_x \cdot n}{T})$ 

8: Step 1: Generate Rhythm Candidates
9: for  $i = 1$  to  $n - 1$  do
10:   for  $j = i + 1$  to  $n$  do
11:      $d \leftarrow x_j - x_i$ 
12:     if  $x_j + \sqrt{n} \cdot d > T + m_x$  then
13:       break
14:     Add candidate with interval  $d$  and anchor  $x_i$  to list

15: Step 2: Count Hits Per Candidate
16: for each candidate  $(d, a)$  do
17:   Walk forward from anchor  $a$  using steps of  $d$ 
18:   At each step  $t_k = a + k \cdot d$ , define expected amplitude  $\bar{y}_k$  from previous hits
19:   Count a hit if  $\exists x_i \in [t_k \pm m_x]$  and  $y_i \in [\bar{y}_k \pm m_y]$ 
20:   Record total hits and tries

21: Step 3: Prune Candidates via Binomial Test
22: for each candidate with hits  $h$  and tries  $t$  do
23:   Compute Bonferroni adjusted threshold  $\alpha' = \alpha/R$ 
24:   Compute  $p$ -value from  $\text{BinomialTest}(h, t, p_{\text{null}})$ 
25:   if  $h < \sqrt{n}$  or  $p\text{-value} \geq \alpha'$  then
26:     Remove candidate

27: Step 4: Group Similar Candidates
28: Construct similarity graph where edge if  $\text{Szymkiewicz-Simpson}(A_i, A_j) \geq \tau$ 
29: for each connected component do
30:   Keep candidate with greatest centrality as dominant
31:   Absorb unique hits from others into dominant candidate's absorbed set

32: Step 5: Final Filtering by Support
33: for each remaining candidate do
34:   if (hits + absorbed hits)  $< \sqrt{n}$  then
35:     Remove candidate
36: return Remaining candidates as final rhythmic intervals
```

Example Walkthrough Using Peak Time Array

We demonstrate the full algorithm using the following observed peak times from an acoustic signal:

$$\begin{aligned} \text{peaks} = \{ &0.1035, 0.3391, 0.5746, 0.8100, 1.0454, 1.2791, 1.5160, 1.7496, \\ &1.9866, 2.2220, 2.4574, 2.6930, 2.9286, 3.1641, 3.3999, 3.6359, \\ &3.8722, 4.1088, 4.3458, 4.5828, 4.8197 \} \end{aligned}$$

The following plot shows the original acoustic signal, with peaks marked:

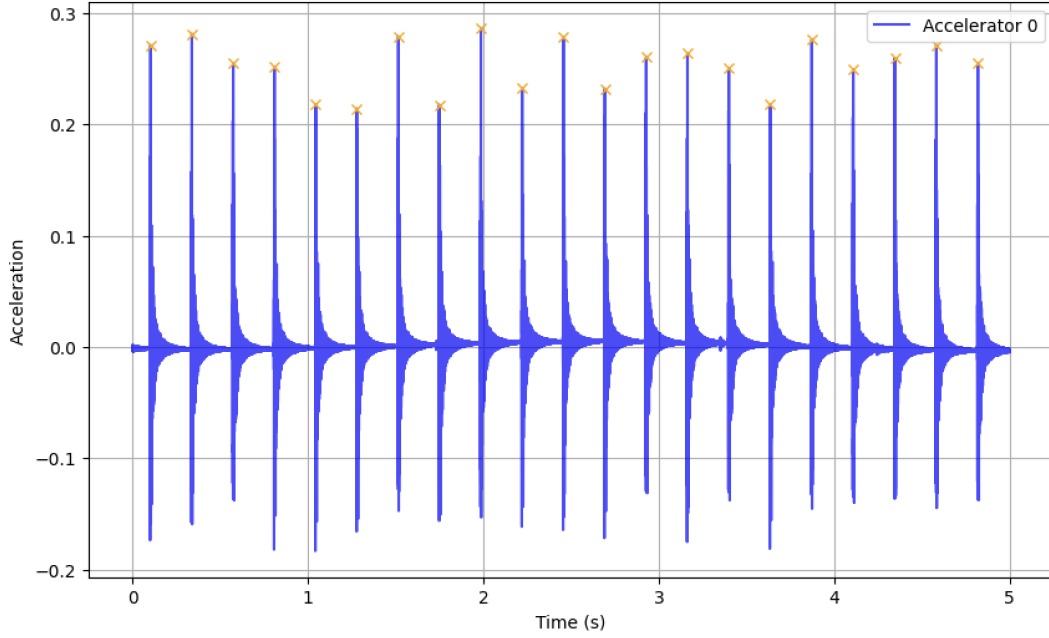


Figure 1: Acoustic signal with extracted peaks

Step 0: Estimate Noise Level

We calculate the global standard deviation of peak-to-peak spacing:

$$\text{SD} = 0.00091 \quad \Rightarrow \quad \text{Margin} = z \cdot \text{SD} = 1.645 \cdot 0.00091 = 0.0015$$

The probability of a random hit in a margin window is estimated as:

$$p_{\text{null}} = \frac{2 \cdot \text{margin} \cdot \# \text{peaks}}{\text{run length}} = \frac{2 \cdot 0.0015 \cdot 21}{5} \approx 0.0126$$

Step 1: Generate Rhythm Candidates

For each peak pair (i, j) , we compute the difference $d = x_j - x_i$ and retain it as a candidate if it allows at least $\sqrt{n}$ steps within the duration.

Example early candidates:

- Candidate 0: $d = 0.2356$, anchor = 0.1035
- Candidate 1: $d = 0.4711$, anchor = 0.1035
- Candidate 2: $d = 0.7065$, anchor = 0.1035

This process generates a total of 41 candidates.

Step 2: Count Hits Per Candidate

We count how many predicted steps align with real peaks (within the x and y margin), starting from each candidate’s anchor and incrementing by d . The first two hits (anchor and anchor+ d) are assumed, and only later steps are evaluated.

Candidate ID	d (sec)	Anchor	Hits	Tries
0	0.2356	0.1035	20	20
1	0.4711	0.1035	10	10
2	0.7065	0.1035	6	6
3	0.9429	0.1035	5	5
4	1.1785	0.1035	3	4

Step 3: Prune Candidates via Binomial Test

Each candidate is evaluated using a one-sided binomial test under:

$$H_0 : \text{hit rate} \leq p_{\text{null}} \quad H_A : \text{hit rate} > p_{\text{null}}$$

A Bonferroni adjustment is applied using α/R for R candidates. Clusters failing the test are removed.

Step 4: Group Similar Candidates

Remaining candidates are grouped based on the Szymkiewicz–Simpson overlap of their hit indices:

$$\text{Sim}(A, B) = \frac{|A \cap B|}{\min(|A|, |B|)}$$

Note that $0 \leq \text{Sim}(A, B) \leq 1$, where a value of 0 indicates that the sets are disjoint, and a value of 1 indicates that the smaller set is entirely contained within the larger. To identify and consolidate redundant candidates, a similarity graph is constructed by connecting pairs of candidates whose overlap exceeds a predefined threshold (e.g., 0.8). Each edge represents significant shared alignment between two candidates. Within each connected component of this graph, the candidate with the highest degree centrality is selected as the representative rhythm. All other candidates in the component are considered redundant and are absorbed into the dominant one, contributing their supporting peak hits.

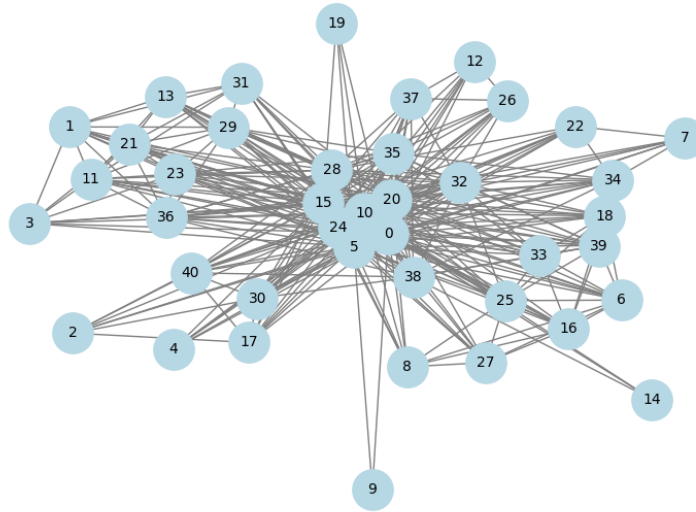


Figure 2: Candidate similarity graph

Step 5: Final Filtering by Support

Candidates with too few total hits (direct + absorbed) are removed. In this example, only rhythms with $\geq \sqrt{21} \approx 5$ hits are retained.

Final Result

The final output is the strongest rhythmic regime:

Detected Rhythm: ID = 0, $\mu = 0.2356$ seconds, aligned with 20 out of 20 peaks

This rhythm is statistically significant, and it absorbs several weaker candidates into a unified regime.

The figure below shows the acoustic signal with peaks labeled according to their assigned rhythmic candidate. Each candidate is represented by a distinct color—if multiple rhythmic candidates are present, the peaks associated with each are shown in different colors.

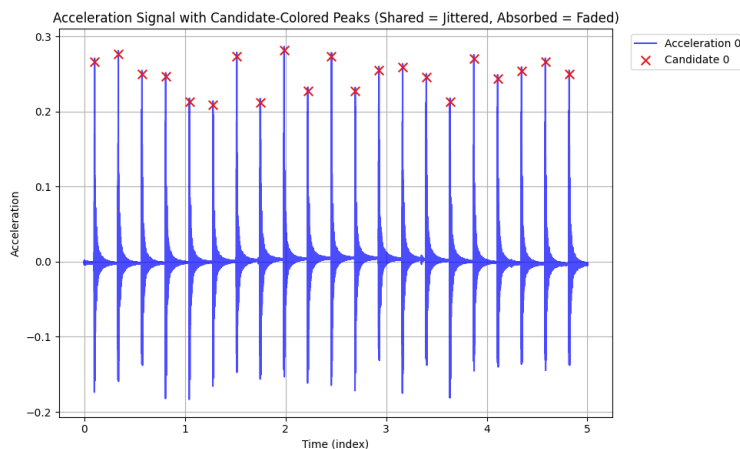


Figure 3: Labeled acoustic signal with candidate assignments

Notes

- Peaks that align with a candidate's predicted steps—but were not part of its original cluster—are marked as absorbed.
- Absorbed peaks are visualized using a lighter shade of the candidate's color to distinguish them from originally clustered peaks.
- Peaks not matched to any candidate rhythm are considered unused and are displayed in gray and likely correspond to random boiling. A higher proportion of unused peaks may indicate random boilings.
- The algorithm reports both the count and proportion of unused peaks, providing a measure of how well the detected rhythms explain the data.